IOT PRACTICUM JOURNAL

MODULE 3

MQTT CONNECTIVITY WITH MICROPYTHON, RASPBERRY PI PICO W, AND ECLIPSE MOSQUITTO

Group : .....

Day/Shift : .....

Instructor Assistant Code : .....

Name : .....

NIM : .....

IOT DEVICE PROGRAMMING PRACTICUM

SCHOOL OF ELECTRICAL ENGINEERING

TELKOM UNIVERSITY

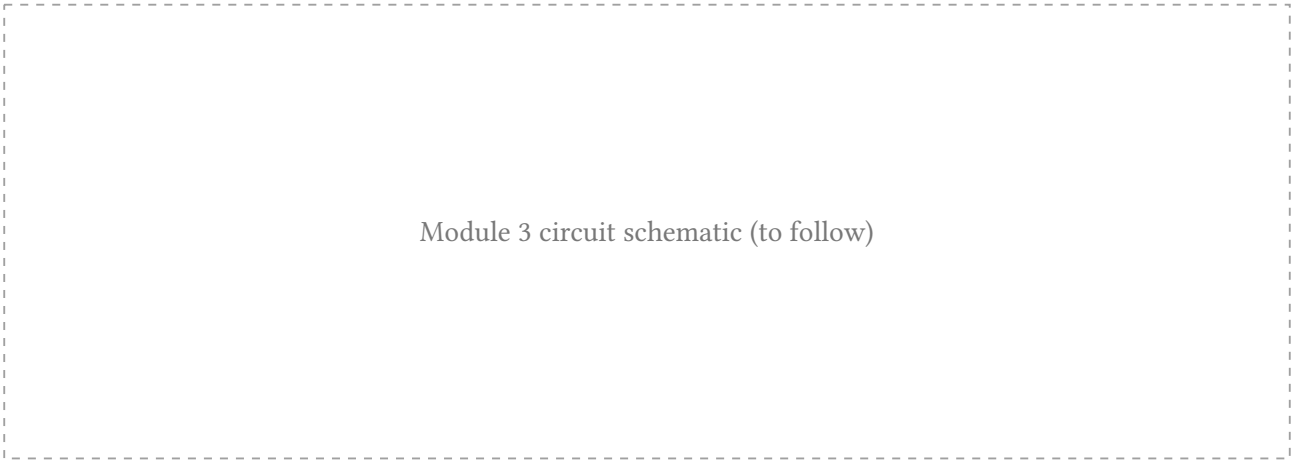
2026

A. Circuit and System

1. Prepare the following components:

   - Raspberry Pi Pico W (2x)
   - Orange Pi 4 PRO (1x, as the Mosquitto broker)
   - 10 kΩ Potentiometer (1x)
   - Breadboard (1x)
   - Wires/Jumpers (as needed)

2. Build the following circuit using the tools and components you have prepared! (POT: Potentiometer)



Publisher (Pico W)		Subscriber (Pico W)
Component	**Pin**	**Component**
POT (VCC)	3V3(OUT)	No additional circuit. The Subscriber only records the values that arrive, so the second Pico W just needs to be on the same Wi-Fi network as the Publisher.
POT (OUT)	GP26 (ADC0)	
POT (GND)	GND	

Note: the Pico W's ADC input only tolerates up to 3.3 V, so the potentiometer's top leg must be fed from 3V3(OUT) (physical pin 36), not from the 5 V VBUS. Its middle leg (the wiper) must land on one of the ADC pins, that is GP26, GP27 or GP28. The Eclipse Mosquitto broker runs on the Orange Pi 4 PRO and needs no additional circuit. Make sure both Pico W boards and the Orange Pi are on the same Wi-Fi network.

Topic assignment per group. Every group uses its own topic, so your group's potentiometer is only recorded by your group's Subscriber. Change `group_id` in **both** programs to your group number; the topic and `mqtt_client_id` follow it automatically.

Group	Topic	**`mqtt_client_id`** *(Publisher)*	**`mqtt_client_id`** *(Subscriber)*
1	`iris/grup1/pot`	`grup1-pub`	`grup1-sub`
2	`iris/grup2/pot`	`grup2-pub`	`grup2-sub`
3	`iris/grup3/pot`	`grup3-pub`	`grup3-sub`
4	`iris/grup4/pot`	`grup4-pub`	`grup4-sub`
5	`iris/grup5/pot`	`grup5-pub`	`grup5-sub`
6	`iris/grup6/pot`	`grup6-pub`	`grup6-sub`

Group	Topic	`mqtt_client_id` *(Publisher)*	`mqtt_client_id` *(Subscriber)*
7	`iris/grup7/pot`	`grup7-pub`	`grup7-sub`
8	`iris/grup8/pot`	`grup8-pub`	`grup8-sub`

An assistant can watch every group at once from the Orange Pi by subscribing to the wildcard topic: `mosquitto_sub -h localhost -t 'iris/+/pot' -v`

3. Before running the program, first prepare the Eclipse Mosquitto broker on the Orange Pi in the following order. Tick every step you have completed!

   - ☐ Installing the packages: `sudo apt update` then `sudo apt install mosquitto mosquitto-clients`
   - ☐ Enabling autostart: `sudo systemctl enable mosquitto.service`
   - ☐ Noting the Orange Pi's IP address with `hostname -I`
   - ☐ Editing `/etc/mosquitto/mosquitto.conf`, adding `listener 1883` and `allow_anonymous true`
   - ☐ Restarting the service: `sudo systemctl restart mosquitto.service`
   - ☐ Installing the `micropython-umqtt.simple` library on both Pico W boards through Thonny
   - ☐ Powering the potentiometer from 3V3(OUT), not VBUS, and wiring its wiper to GP26

   Note: `allow_anonymous true` may only be used on an isolated practicum network. If your assistant asks for a broker with authentication, use `sudo mosquitto_passwd -c /etc/mosquitto/passwd <USERNAME>` and then fill in `mqtt_username` and `mqtt_password` in both program files.

4. Complete the program below by filling in the blank sections. Afterwards (or while completing it), retype the blank sections you have filled in using the Thonny application prepared by your respective lab assistants!

   Blank sections are marked with an underscore run `_____` and numbered `(1)` through `(6)` in the comment to their right. Replace each underscore run with the correct code following the hint in its comment. Note that the **topic** value must match exactly on the Publisher and the Subscriber, while `mqtt_client_id` must differ for every device.

Publisher Source Code — MQTT (MicroPython, Pico W)

```
49  sample_count = 0
50  last_percent = -100
51
52  # The Pico W is configured as a station (client) joining an Access Point.
53  wlan = network.WLAN(network._____)  # (1) Station (client) mode constant
54  wlan.active(True)
55  wlan.connect(wifi_ssid, _____)  # (2) Wi-Fi password variable
56
57  while not wlan.isconnected():
58      print('Waiting for connection...')
59      time.sleep(1)
…
99  # broker to expect a ping from this client every 30 seconds.
100 mqtt_client = MQTTClient(
101     client_id=mqtt_client_id,
102     server=mqtt_host,
103     port=_____,  # (3) The listener value in mosquitto.conf
104     user=mqtt_username,
105     password=mqtt_password,
106     keepalive=30,
```

```
)
…
try:
    while True:
        try:
            raw = pot._____()  # (4) 16-bit ADC read method (0 through 65535)
            percent = round(raw * 100 / 65535)

            if abs(percent - last_percent) >= _____:  # (5) Change threshold variable
                sample_count += 1
                last_percent = percent

                # The message carries a sample number so a line in the Publisher
                # Shell can be paired one-to-one with a line in the Subscriber Shell.
                payload = "{} {}".format(sample_count, percent)

                print(f'[{timestamp()}] Position {percent} %, publish #{sample_count}')
                mqtt_client._____(mqtt_publish_topic, payload)  # (6) Send method

            # umqtt.simple does not ping on its own; without a periodic ping
            # here, an idle connection can drop silently and the next publish
            # fails without any warning at all.
```

Subscriber Source Code — MQTT (MicroPython, Pico W)

```
import network
import time
import ntptime
import utime
from umqtt.simple import _____  # (1) Name of the MQTT client class used below

wifi_ssid     = "<YOUR SSID WIFI>"
wifi_password = "<YOUR WIFI PASSWORD>"
…
# topic and the client id follow it automatically, so this Subscriber only
# records your group's potentiometer.
group_id = 1

mqtt_receive_topic = "iris/grup{}/pot".format(_____)  # (2) Group number variable
mqtt_client_id     = "grup{}-sub".format(group_id)  # Must be unique per client

# Time zone offset from UTC in seconds. WIB (UTC+7) = 7 * 3600.
timezone_offset = 7 * 3600
…
def mqtt_subscription_callback(topic, message):
    # The receive time is recorded first so nothing else can delay it.
    received_at = _____()  # (3) The wall-clock function defined above

    sample, percent = message.decode().split()

    print(f'[{received_at}] Topic {topic.decode()} received sample #{sample}'
          f' | Position: {percent} %')


# The callback must be registered before connecting to the broker.
mqtt_client.set_callback(_____)  # (4) Callback function name, without parentheses
mqtt_client.connect()

mqtt_client._____(mqtt_receive_topic)  # (5) Method that registers the subscription
print("Connected and subscribed")

last_ping = utime.ticks_ms()
…
        try:
            # check_msg() is non-blocking, so this loop still gets to send its
            # periodic ping; wait_msg(), which blocks forever, would never come
            # back to do that.
            mqtt_client._____()  # (6) Message-checking method (non-blocking)

            if utime.ticks_diff(utime.ticks_ms(), last_ping) > 20000:
                mqtt_client.ping()
                last_ping = utime.ticks_ms()
```

5. Record the configuration you used in the following table! Note that the **topic** must match exactly on both devices, while `mqtt_client_id` must instead differ; the broker will drop one of the clients if both use the same ID.

Parameter	**Value used**
Broker address (`mqtt_host` / `hostname -I`)	
Broker port (`listener` in `mosquitto.conf`)	
Topic (`mqtt_publish_topic` = `mqtt_receive_topic`)	
Publisher `mqtt_client_id`	
Subscriber `mqtt_client_id`	
Potentiometer pin on the Pico W (`ADC`)	
Change threshold (`deadband_pct`)	

B. Analysis

1. Draw the communication flow implemented in the system you have worked on!

2. Next, a response-time experiment will be carried out for messages sent from the **Publisher** to the **Subscriber** through the MQTT broker. First open **TWO (2)** Thonny Shells/REPLs to observe the output of the **Publisher** and the **Subscriber**. Turn the potentiometer knob slowly: every time its position shifts past `deadband_pct`, the Publisher Shell prints the **publish time** along with the sample number and the position, while the Subscriber Shell prints the **receive time** along with the same sample number and position. That sample number is what pairs a line in one Shell with a line in the other. Record:
   - **FIVE (5)** pairs of readings: the publish time, sample number and position from the Publisher Shell, together with the receive time from the Subscriber Shell

   Both times are recorded in **HH:MM:SS:MS** format. Compute the **one-way delay** by subtracting the publish time from the receive time.

   Note: both Pico W boards synchronise their clocks over NTP (`ntptime.settime()`) as soon as they connect to Wi-Fi, then lock a single millisecond reference on an RTC second rollover. That means times from the two Shells really can be subtracted and the result is a one-way delay through the broker. Without that synchronisation step, a Pico W's RTC starts at 1 January 2021 and the two boards would never agree on the

time. NTP accuracy over Wi-Fi is around 10–50 ms. This module has no round-trip column because MQTT is one-way here: the Subscriber only records and echoes nothing back.

Trial no. …	Sample no.	Position **[%]**	Publish time *(Publisher)* **[HH:MM:SS:MS]**	Receive time *(Subscriber)* **[HH:MM:SS:MS]**	One-way delay **[ms]**
1.					
2.					
3.					
4.					
5.					
Average one-way delay (ms)					

3. Write your **analysis** of the system you experimented with earlier in the fill-in section below!

(Also give the **relationship/correlation** by applying the **communication methodology** you are studying in this module; it does not have to be complete!)

C. Suggestions and Feedback

1. Assistant

2. Laboratory